

# Classifying Cars to Assist ALPR Systems

---

presented by: Sem Verkerk and Tyler Fabela

---

CS 171-01

---

---



# The amount of cars the police have to monitor is increasing...

---

How can we improve upon existing systems to help Police Officers who are struggling to keep their eyes on everything all at once?



# Dataset

## Description

This dataset contains handpicked photos of the exteriors of all kinds of cars. The cars differ in color, make, model, and year. There are no duplicates of images but there are multiple pictures of the same car.

## Size

60,000 handpicked photos  
~780 MB

## Type

JPG  
RGB images of vehicles

# Preprocessing

01

## Parse Dataset

- What kind of images do we have?
- Which images do we need to clean up?

02

## Create Model

After identifying how the dataset needs to be cleaned we created a model to clean the data.

03

## Clean Data

Use the new model to clean the dataset and shrink the dataset from 60,000 images to 10,000 images.

04

## Analyze Results

We had to analyze the new shrunked datasets to make sure the model accomplished what we wanted it to do.

# Data Split

We split our 10,000 image dataset into three categories:

Training

80%

7400 images

Validation

10%

925 images

Testing

10%

925 images

# Custom Model

## Model details:

### Architecture

Convolutional Neural  
Network

### Structure

- 512 conv layers
- Batch Normalization
- Dropout 0.1  $\rightarrow$  0.3
- 512 dense layer units
- Global Average Pooling
- Softmax output

### Regularization

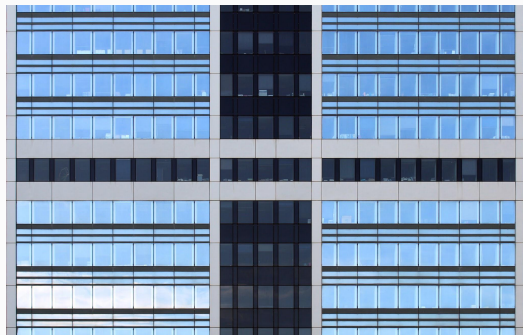
L2 Regularization  
Dropout (up to 0.5)



02

# Pre-Trained Model

Accuracy was performing as poorly—if not worse—than the custom CNN:



01

Needed to configure model in order to address pre trained CNN's underfitting (both testing and validation accuracies were always together)



02

Initially had L2 regularization too high (worried about constant overfitting from custom model); we needed to be less cautious about penalizations.



03

Resolved issue of underfitting at the cost of having pretrained model overfit as well

# Training Setup



## — Setup

- Optimizer: Adam (lr = 0.001)

Loss: Categorical  
Cross-entropy + label  
smoothing (0.05)

Epochs: 50

Batch Size: 64

## — Callbacks Used

- EarlyStopping (patience 8)

ReduceLROnPlateau

ModelCheckpoint

## — Environment

- For GPU: Google Colab

Used L4 GPU for faster  
computations (bit the  
bullet and paid for \$10  
worth of compute units)





# Experiments

To improve our model's accuracy we played around with the following settings:

Convolution layers, regularizers, batch numbers, image sizes, data augmentation, dropout, heads, and more...

- Each of these impacted whether the model under fitted or overfitted drastically
- Needed to allow the model to understand patterns without memorization (used L2 regularization and dropouts)

# Training Results

## Custom Model

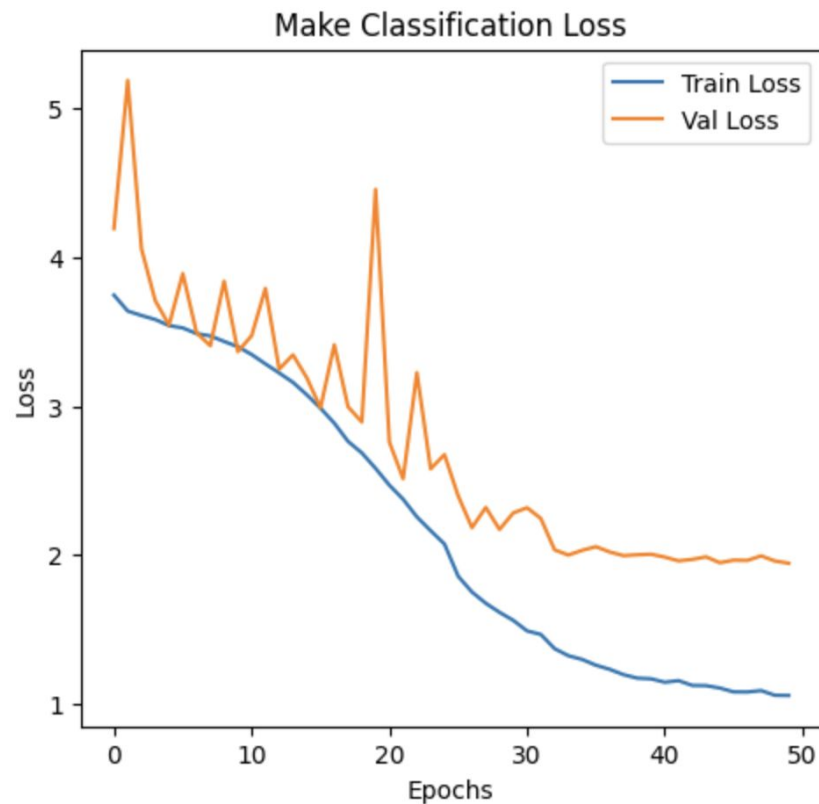
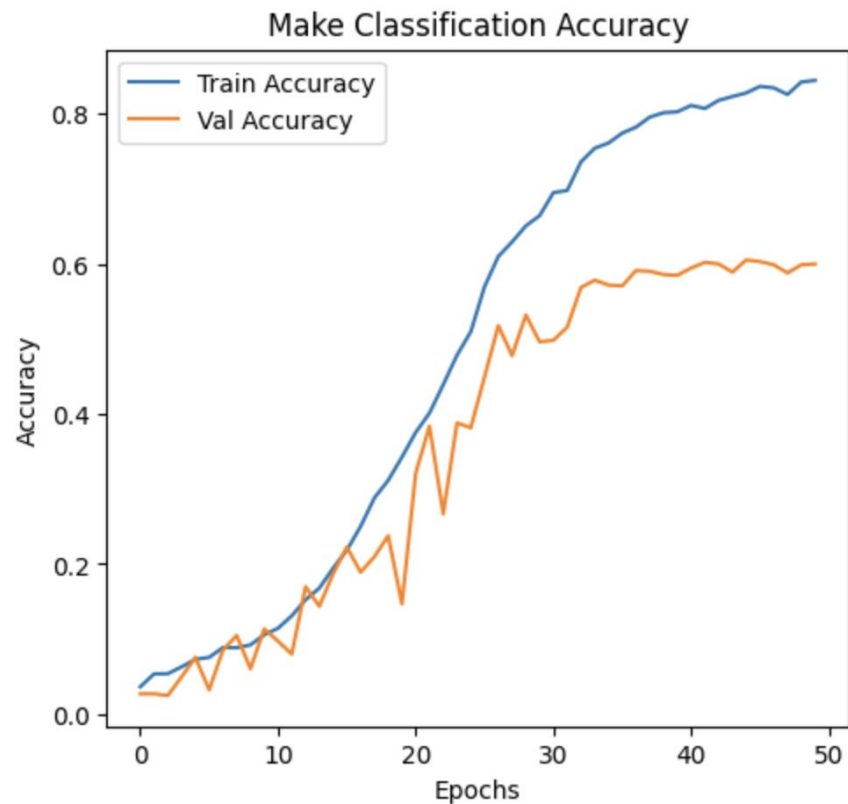
- Since there were too little images for each class—and there are a lot of classes for the model to classify—the custom model would underfit if regularization was too high, resulting in training and validation/testing accuracies of 60% and 40% or worse. However, if we were too lenient
  - The main issue was that tweaking the regularization even a little bit would cause the model to flip between overfitting and underfitting
- We decided to stick with the overfitted model since it could at least recognize the augmented training data and have a higher testing accuracy rather than an under fitted model doing a 50/50 guess for everything (both training and testing).

## Pretrained model

- Similar to the custom model, the pretrained model was sensitive to regularization tweaks, but the overfitted model had an overall higher accuracy.

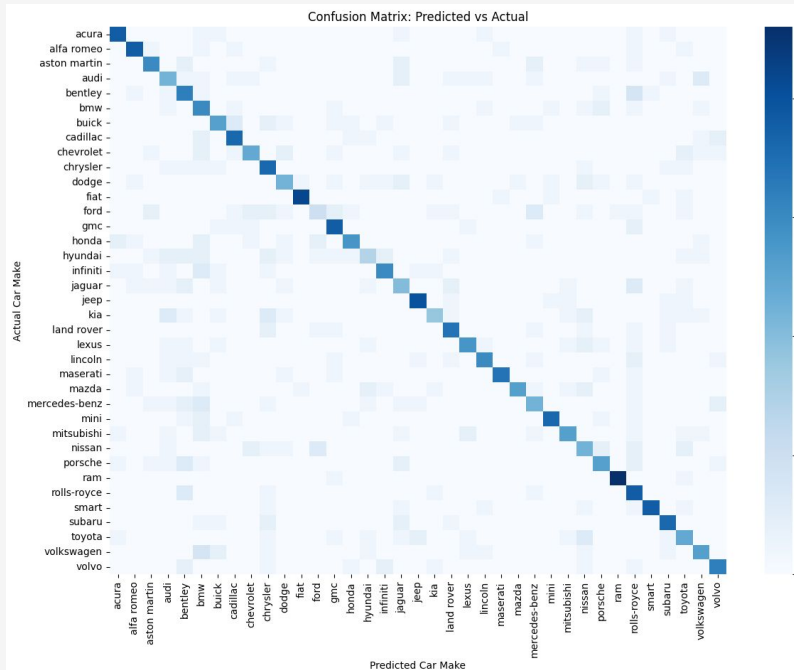
|               | precision | recall | f1-score | support |
|---------------|-----------|--------|----------|---------|
| acura         | 0.82      | 0.72   | 0.77     | 25      |
| alfa romeo    | 0.59      | 0.64   | 0.62     | 25      |
| aston martin  | 0.57      | 0.64   | 0.60     | 25      |
| audi          | 0.38      | 0.48   | 0.42     | 25      |
| bentley       | 0.78      | 0.72   | 0.75     | 25      |
| bmw           | 0.41      | 0.56   | 0.47     | 25      |
| buick         | 0.74      | 0.68   | 0.71     | 25      |
| cadillac      | 0.69      | 0.88   | 0.77     | 25      |
| chevrolet     | 0.64      | 0.56   | 0.60     | 25      |
| chrysler      | 0.65      | 0.80   | 0.71     | 25      |
| dodge         | 0.62      | 0.60   | 0.61     | 25      |
| fiat          | 0.76      | 0.76   | 0.76     | 25      |
| ford          | 0.42      | 0.52   | 0.46     | 25      |
| gmc           | 0.74      | 0.68   | 0.71     | 25      |
| honda         | 0.68      | 0.84   | 0.75     | 25      |
| hyundai       | 0.38      | 0.44   | 0.41     | 25      |
| infiniti      | 0.68      | 0.60   | 0.64     | 25      |
| jaguar        | 0.43      | 0.40   | 0.42     | 25      |
| jeep          | 0.79      | 0.88   | 0.83     | 25      |
| kia           | 0.61      | 0.56   | 0.58     | 25      |
| land rover    | 0.54      | 0.60   | 0.57     | 25      |
| lexus         | 0.53      | 0.72   | 0.61     | 25      |
| lincoln       | 0.70      | 0.64   | 0.67     | 25      |
| maserati      | 0.78      | 0.56   | 0.65     | 25      |
| mazda         | 0.74      | 0.56   | 0.64     | 25      |
| mercedes-benz | 0.32      | 0.36   | 0.34     | 25      |
| mini          | 0.82      | 0.56   | 0.67     | 25      |
| mitsubishi    | 0.71      | 0.48   | 0.57     | 25      |
| nissan        | 0.37      | 0.44   | 0.40     | 25      |
| porsche       | 0.60      | 0.36   | 0.45     | 25      |
| ram           | 0.83      | 1.00   | 0.91     | 25      |
| rolls-royce   | 0.69      | 0.80   | 0.74     | 25      |
| smart         | 0.95      | 0.72   | 0.82     | 25      |
| subaru        | 0.46      | 0.44   | 0.45     | 25      |
| toyota        | 0.67      | 0.48   | 0.56     | 25      |
| volkswagen    | 0.73      | 0.44   | 0.55     | 25      |
| volvo         | 0.61      | 0.68   | 0.64     | 25      |

# Test Results

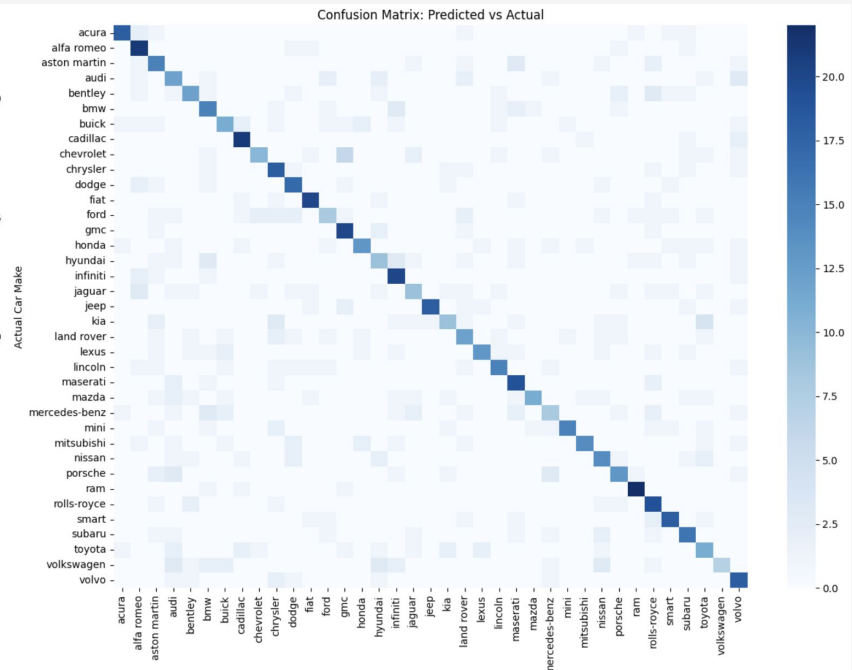


# Predictions / Visual Results

## Custom Model



## Pretrained Model



# Model Comparison



|           | Vers 1           | Vers 2           | Vers 3           | Pretrained       |
|-----------|------------------|------------------|------------------|------------------|
| Accuracy  | ~62%, ~39%, ~38% | ~71%, ~59%, ~59% | ~85%, ~61%, ~62% | ~90%, ~70%, ~70% |
| Precision | ~36%             | ~60%             | ~63%             | ~68%             |
| Recall    | ~37%             | ~55%             | ~62%             | ~69%             |
| F1 Score  | ~35%             | ~56%             | ~62%             | ~69%             |

# Challenges / Limitations

## Challenge 01

We had to figure out how to effectively and efficiently clean the dataset because we had too many images, and some of them were undesirable.



## Challenge 02

To raise training and test accuracy we had to play around with the custom model to make sure that we were getting desirable results (time consuming).



## Challenge 03

250 images was definitely too little for a custom CNN to classify a car make, let alone 40 different classes.



## Challenge 04

The dataset was too small and the number of classes was too large to get either the custom or pretrained model to learn the features (>60% acc.) without overfitting during training (>20% diff.).



# Conclusion

We found that it's quite hard to train a model with limited resources and we have a better understanding of why such huge datasets and data centers are necessary for model growth.

---